Entwurf digitaler Systeme

Labor

Ausgabe 3.4.1, 11.02.2026 Basiert auf Basys 3 FPGA Board von Digilent
Autor: Klaus Gosger

Änderungen gegenüber 3.4: Rechtschreibfehler korrigiert

Vorwort

Dieses Skript soll als Anleitung zum (Selbst-)Studium bzw. zur Begleitung des Labors EDS dienen. Ein Lehrbuch über Systementwurf mit VHDL oder eine Anleitung zur im Labor Verwendeten Software und Hardware kann und soll es nicht ersetzen, sondern vielmehr Anregungen geben, sich mit den Fragen der gegebenen Materie auseinanderzusetzen.

Beispiele und Lösungen wurden aus didaktischen Gründen oft vereinfacht und auch nicht immer komplett, in manchen Fällen sogar bewusst fehlerhaft aufgeführt.

Generell ist das Skript auch nach fast 10 Jahren sicher noch nicht fehlerfrei – für Hinweise auf eventuelle Fehler bin ich dankbar.

Inhaltsverzeichnis

Versuch 1 - Schachuhr 4
1.1 Konzeption der Schaltung 5
1.2 Schaltungsentwurf 6
1.3. Funktionale Verifikation 7
1.4. Schaltungssynthese und Implementierung 8
Versuch 2 - freies Thema 9
2.1. Funktionale Beschreibung 9
2.2. Schaltungsentwurf 9
2.3. Funktionale Verifikation 9
2.4. Schaltungssynthese 9
3. Zielsystem 10
Anhang 1: Lösungen zu den Fragen und Beispiele für VHDL 12
A.1 Schachuhr 12
Anhang 2: Literatur 21

Versuch 1 - Schachuhr

Mit Hilfe einer Baugruppe zur Evaluierung soll eine Art Schachuhr auf einem FPGA (Field Programmable Gate Array) realisiert werden. Zum Einsatz kommt das Basys 3™ FPGA Board der Firma Digilent mit dem FPGA **XC7A35T- 1CPG236C** der Firma Xilinx.

Bild 1: Basys 3 Board:

Die Schachuhr soll folgende Funktionen implementieren:

(1) Die Schachuhr soll die verstrichen Zeit in Minuten und Sekunden pro Spieler für insgesamt 2 Spieler messen. Dazu darf diese im Gegensatz zu wirklichen Schachuhren von 0 an nach „Oben“ Zählen.

(2) Umschalten der Zeitanzeige von Spieler A auf Spieler B bzw. zurück nach jedem Schachzug (durch den am Zug befindlichen Spieler),

(3) Anzeige der insgesamt benötigten Zeit für den am Zug befindlichen Spieler in Minuten und Sekunden,

(4) Anhalten und Fortsetzen der Uhr bei Spielunterbrechungen (Start/Stopp-Funktion),

(5) Reset-Funktion, also zurücksetzender Zeiten für beide Spieler auf 0.

Zur Anzeige sollen die auf der Baugruppe befindlichen Bauteile verwendet werden (7-Segmentanzeige mit 4 Stellen, Schalter bzw. Taster, 100 MHz Taktsignal). Bitte beachten Sie, dass Taster eventuell entprellt werden müssen, näheres hierzu finden Sie in Anhang 2.

Bitte beachten Sie auch die Hinweise von Digilent zum BASYS3 Board im entsprechenden Reference Manual [5]. Hier finden Sie viele Anwendungshinweise, u.a. auch zur Programmierung des FPGAs auf dem Board mit Vivado. Diese Hinweise sind in diesem Skript NICHT nochmal aufgeführt!

1.1 Konzeption der Schaltung

Aufgabe 1.1: Zeichnen Sie ein Blockdiagramm der Schachuhr, das in einer HDL-Beschreibung die Entity darstellt.

Aufgabe 1.2 Erstellen Sie ein Konzept bzw. grobes Blockschaltbild für das Innenleben: Welche Funktionen bzw. Komponenten benötigen Sie (z.B. Sekundentakt, Speicher bzw. Zähler für die Spielzeiten, Steuerung durch Tasten, Anzeige etc)?
Hinweis: Wählen Sie einen Top-Down-Ansatz (erst die Struktur, die Details später). Ignorieren Sie zunächst bitte Reset und Start/Stopp. Abstrahieren Sie die Ansteuerung der 7-Segmentanzeige zu einem Funktionsblock Anzeige, der später detailliert werden kann.

Aufgabe 1.3: Ergänzen Sie das Blockdiagramm um die Funktionen Reset und Start/Stopp. Begründen Sie Ihren Entwurf.

Aufgabe 1.4: Detaillieren Sie die Speicher bzw. Zähler für die Spielstände.

Aufgabe 1.5: Detaillieren Sie den Funktionsblock für die 7-Segmentanzeige so, dass hervorgeht, wie Sie die 4 Ziffern der Sieben-Segment-Anzeigen ansteuern.

Hinweise über die Verwendung der 7-Segmentanzeige finden Sie im Reference Manual zum Basys 3 Board. Folgendes Prinzip kommt zur Anwendung:

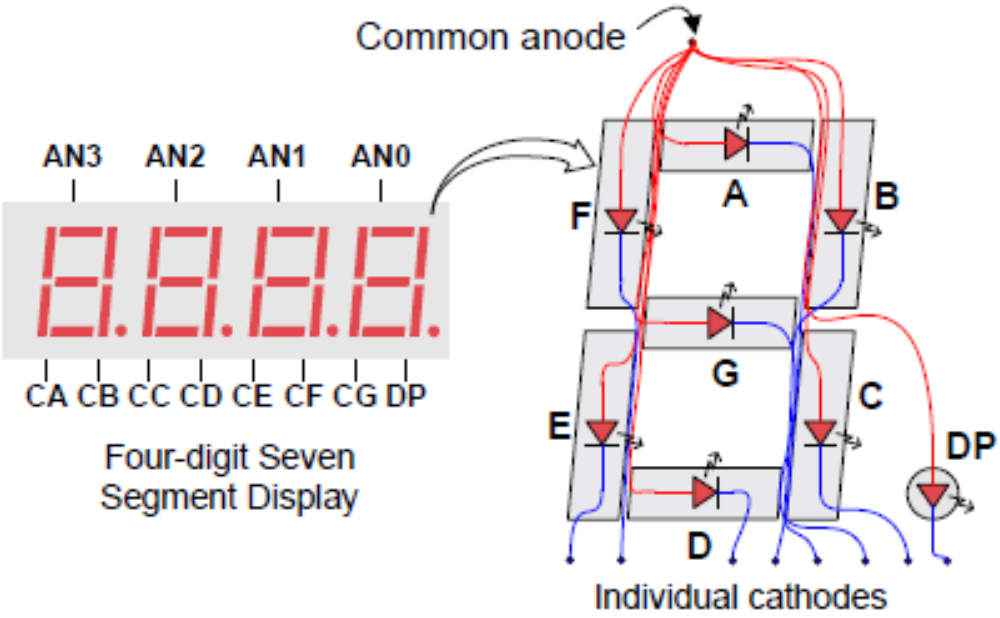


Bild 2: Basys 3 Board: Verschaltung 7-Segment -Anzeige

Die 4 Anzeigen (Ziffern) werden gemultiplext um Anschlüsse zu sparen. Jede Anzeige kann über ein Enable-Signal (Common Anode) separat ein- und ausgeschaltet werden. Um verschiedene Ziffern auf den 4 Segmenten darzustellen, müssen diese nacheinander angesteuert werden, das Prinzip erkennen Sie aus nachfolgender Darstellung. Weitere Details entnehmen Sie bitte dem Reference Manual.

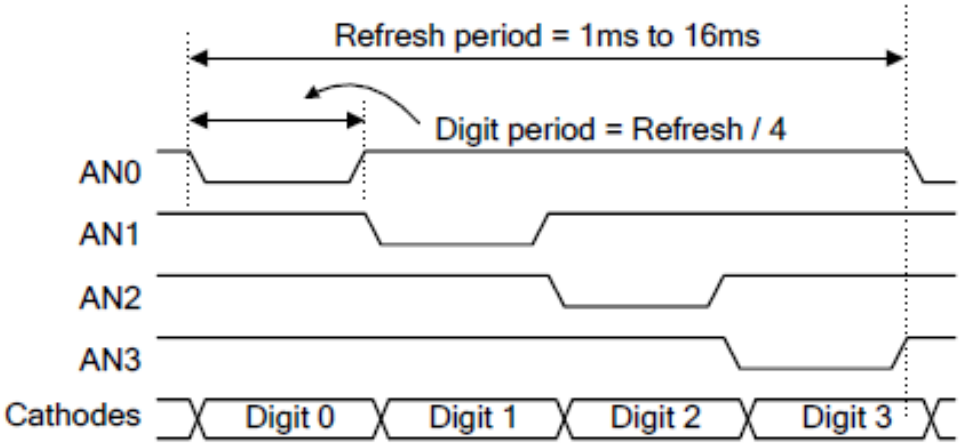


Bild 3: Beispiel für Timing von gemultiplexten 7-Segment-Anzeigen

1.2 Schaltungsentwurf

Die in der Konzeption festgelegten Funktionsblöcke sind nun als Schaltung zu entwerfen, d.h. in VHDL zu beschreiben. Für den Entwurf können sie die Muster aus den angehängten Lösungen als Basis verwenden.

Hinweis: Wenn Sie in zu zweit bzw. in Gruppen organisiert sind, können Sie die Aufgaben folgendermaßen aufteilen und **parallel** starten:

(1) Schaltungsentwurf und funktionale Verifikation: machen Sie sich mit der Funktion der Schaltung und der Simulation der Schaltung vertraut, damit Sie auf Tests und mögliche Fehler bei der Implementierung vorbereitet sind.

(2) Schaltungssynthese: Machen Sie sich mit den Synthesewerkzeugen vertraut. Verwenden Sie hierzu die vorgegebenen Muster als Dateivorlagen für den Umgang mit den Werkzeugen zur Synthese und Implementierung

Aufgabe 1.6: Entwerfen Sie die Schaltung in VHDL. Als Vorlage können Sie die HDL-Beschreibung im Skript verwenden. Analysieren Sie die Vorlage und **ändern** Sie gegebenenfalls nach Bedarf.

1.3. Funktionale Verifikation

Übernehmen Sie die Schaltungsentwürfe für die Schachuhr in den HDL-Simulator und übersetzen Sie den Schaltungsentwurf. Übersetzen Sie die Dateien zunächst einzeln. Arbeiten Sie bitte direkt auf dem Verzeichnis, auf dem Sie die VHDL Dateien abgelegt haben, so dass bei der Fehlerkorrektur dort der fehlerfreie Quellcode abgelegt bleibt.

Aufgabe 1.7: Übersetzen Sie den Schaltungsentwurf und beheben Sie alle ggf. auftretenden Fehler, die der Compiler meldet.

Überlegen Sie sich eine Methode, um den Schaltungsentwurf zu verifizieren. Hierzu benötigen Sie eine Testumgebung, die alle Eingangssignale Ihrer Schaltung stimuliert und die Ausgangssignale aufnimmt. Hinweis: eventuell ist es hilfreich, die Signale im Prüfling zum Test hinzuzuziehen.

Aufgabe 1.8: Entwerfen Sie eine Testumgebung für ihre Schaltung.

Aufgabe 1.9: Testen Sie das Modul im Simulator. Analysieren Sie die Ausgangssignale des Prüflings (DUT). Prüfen Sie die Ergebnisse auf Plausibilität. Machen Sie notwendige Änderungen und wiederholen die Aufgaben 1.6 bis 1.9. bis sie einen verifizierten Schaltungsentwurf haben.

1.4. Schaltungssynthese und Implementierung

Setzen Sie sich mit den Werkzeugen zur Synthese und Implementierung auseinander. Stellen Sie den Zusammenhang mit dem in der Abbildung gezeigten grundsätzlichen Ablauf her.

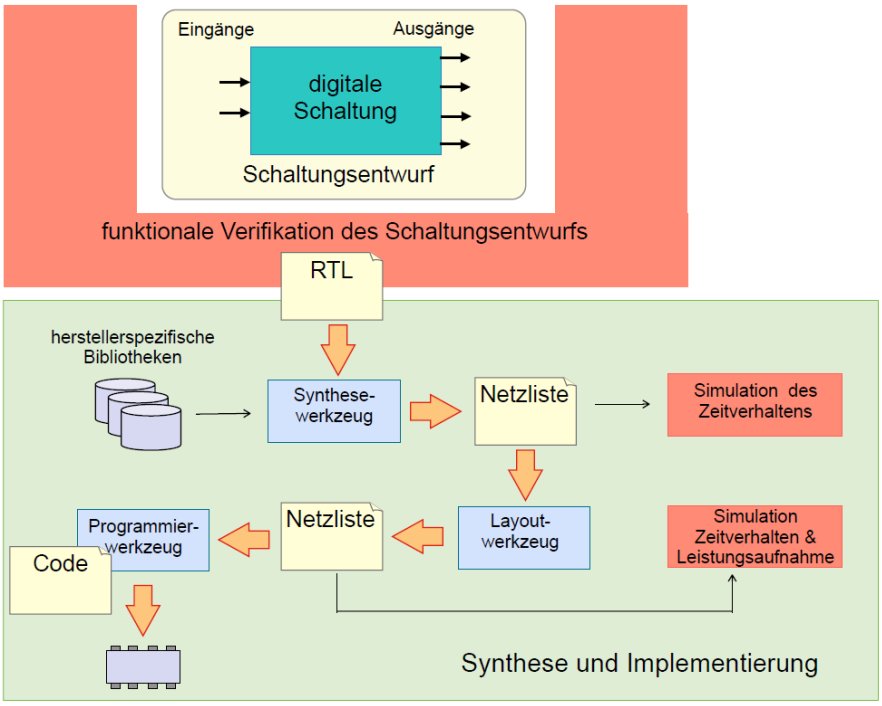


Bild 4: Synthese und Implementierung des Schaltungsentwurfs

Aufgabe 1.11: Machen Sie sich mit dem Synthesewerkzeug (Vivado Design Suite) vertraut. Verwenden Sie hierzu die vorgegebenen Muster als Dateivorlagen für den Umgang mit den Werkzeugen zur Synthese und Implementierung. Eine Anleitung zum Vorgehen finden Sie hier Vorgehen siehe hier: https://digilent.com/reference/programmable-logic/guides/installing-vivado-and-vitis

Das Entwicklungswerzeug Vivado Design Suite von Xilinx finden Sie hier (Installation via Self Extracting Web Installer); bitte darauf achten, dass die von Ihnen gewählte Version auch den Artix 7 FPGA unterstützt:
https://www.xilinx.com/support/download.html

Aufgabe 1.12: Überprüfen Sie die Ergebnisse der Verifikation des Schaltungsentwurfs (Dokumentation zu Aufgabe 1.9). Synthetisieren und implementieren Sie die den Schaltungsentwurf und testen Sie seine Funktionen auf dem Basys 3 Board. Wiederholen Sie gegebenfalls Schritte, bis die Schaltung das gewünschte Verhalten zeigt. Passen Sie ihre Dokumentation gegebenfalls an.

Versuch 2 - freies Thema

Als weiterer Versuch ist Ihnen ein Thema freigestellt. Ausgehend von Versuch 1 (Schachuhr), kämen beispielsweise folgende Abwandlungen / Ergänzungen in Frage: (1) Stoppuhr / Eieruhr oder (2) ein Metronom (Anzeige „beats per minute“) usw...

Ihr Thema kann aber auch in eine völlig andere Richtung gehen, z.B. ein Signalgenerator (siehe Vorlesungsmanuskript) oder ein VGA Schnittstellen-Treiber oder ein Lauflicht / Laufschriftleiste...

Die Art des Themas richtet sich nach Ihren eigenen Interessen, sowie nach den Möglichkeiten der verwendeten Entwicklungsumgebung und des Zielsystems. Beispielsweise können Sie sich nach frei verfügbaren Bausteinen (IP-Cores) umschauen, bzw. ein komplett eigenes Design realisieren.

Gehen Sie bitte wie folgt vor:

- Wenn möglich, wählen Sie sich einen Sparringspartner in Ihrem Kurs, mit dem Sie Ihr Thema gemeinsam realisieren. Auf diese Weise können Sie Arbeiten aufteilen und den Fortschritt und das weitere Vorgehen miteinander diskutieren. Da die Synthesewerkzeuge recht komplex sind, ist beispielsweise eine Arbeitsaufteilung in (1) Entwurf und Test der Schaltung, sowie (2) Synthese und Implementierung denkbar.
- Besprechen Sie Ihren Themenvorschlag mit Ihrem Betreuer. Bei dieser Gelegenheit ist auch zu klären, ob Sie zusätzliche Hardware benötigen und anschaffen möchten.
- Realisieren Sie Ihren Systementwurf in den in den folgenden Kapiteln angegebenen Schritten.
- Dokumentieren Sie die wichtigsten Ergebnisse in jedem der Schritte.
- Präsentieren Sie Ihren Systementwurf in der Abschlusspräsentation vor den anderen Teilnehmern des Kurses. (Dieser Punkt kann je nach Zeiterfordernissen entfallen)

Das Testat für das Labor erhalten Sie nach Abgabe der Dokumentation (Laborbericht) zu diesem freien Thema bei Ihrem Betreuer.

2.1. Funktionale Beschreibung

Beschreiben Sie Ihr Vorhaben und das Funktionsprinzip. Als Muster für die Dokumentation können Sie die Pflichtaufgabe verwenden, bzw. auf Literatur oder Fundstellen im Web verweisen. Beschreiben Sie Ihre geplante Vorgehensweise und ggf. die Aufteilung der Aufgaben.

2.2. Schaltungsentwurf

Entwerfen Sie Ihre Schaltung mit VHDL. Dokumentieren Sie Ihre Ergebnisse.

2.3. Funktionale Verifikation

Schreiben Sie eine Testumgebung zu Ihrem Schaltungsentwurf und testen Sie die Schaltung. Dokumentieren Sie Ihre Ergebnisse. Sollte eine Verifikation mittels Testumgebung und Simulation nicht sinnvoll möglich sein, ist trotzdem eine Dokumentation der Testfälle erforderlich. Sinnvoll ist hier z.B. eine Tabelle mit den durchgeführten Testfällen und Ergebnissen der Tests.

2.4. Schaltungssynthese

Implementieren Sie Ihren Schaltungsentwurf auf der Ziel-Hardware mit Hilfe der Entwicklungsumgebung. Dokumentieren Sie Ihre Ergebnisse.

3. Zielsystem

Als Zielsystem kommt ein Evaluations-Board von Digilent zum Einsatz. Dieses Board bietet bereits umfangreiche Schnittstellen auf der Leiterkarte, die Sie in ihrem Versuch verwenden können.

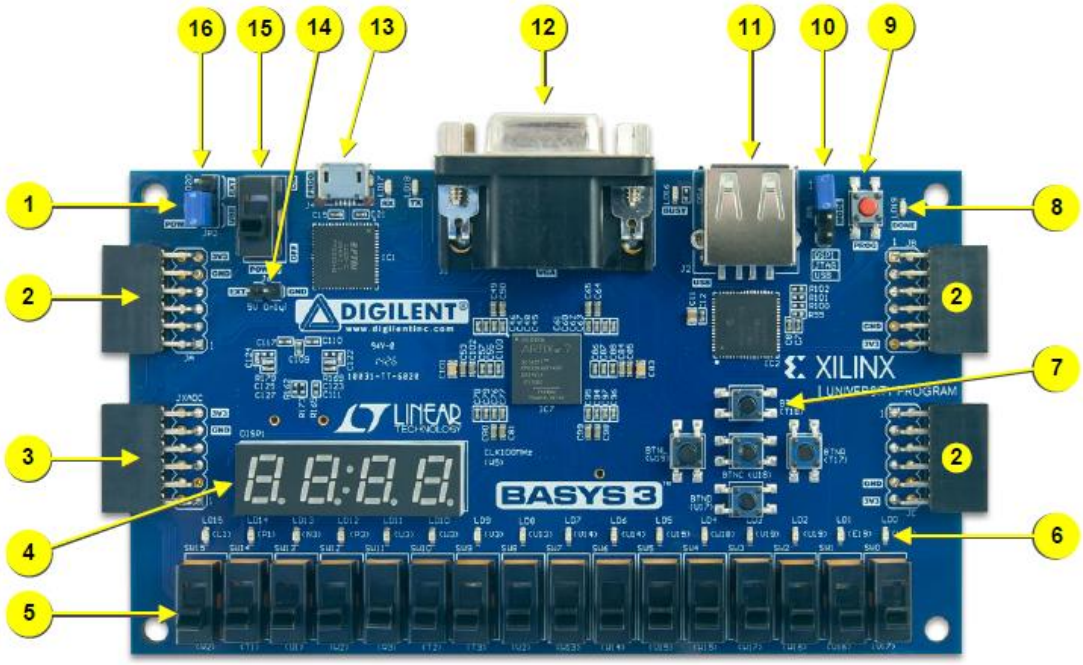


Figure 1. Basys 3 FPGA board with callouts.

Callout	Component Description	Callout	Component Description
1	Power good LED	9	FPGA configuration reset button
2	Pmod port(s)	10	Programming mode jumper
3	Analog signal Pmod port (XADC)	11	USB host connector
4	Four digit 7-segment display	12	VGA connector
5	Slide switches (16)	13	Shared UART/ JTAG USB port
6	LEDs (16)	14	External power connector
7	Pushbuttons (5)	15	Power Switch
8	FPGA programming done LED	16	Power Select Jumper

Table 1. Basys 3 Callouts and component descriptions.

Bild 5: Basys 3 Board – vorhandene Peripherie

Außer dem FPGA-Baustein (Xilinx Artix 7 XC7A35T- 1CPG236C) und der zugehörigen fest eingebauten Peripherie (Arbeitsspeicher, Flash-Speicher, LED- und 7-Segment-Anzeigen, Mikroschalter, Taster und Taktgenerator) sind folgende Schnittstellen zur Erweiterung bzw. zur Kommunikation vorhanden:

- Steckplätze für vier I/O Erweiterungen (6x2) mit insgesamt 32 IOs, davon einer für Analoge
- USB-Port
- PS/2 Port und VGA Port

Der USB-Port dient der Kommunikation mit einem Rechner. Hierunter fällt auch die Programmierschnittstelle für die Entwicklungswerkzeuge. Ebenfalls über den USB-Port erfolgt die Stromversorgung.

Die gesamte Peripherie ist an die FPGA Pins angeschlossen (die genaue Belegung finden Sie im User Manual bzw. im Master XDC File). Das Master XDC File dient der Festlegung der Constraints beim Entwurf in Vivado, also insbesondere der Festlegung welche Ports ihrer VHDL Module mit welchen Pins des FPGA verbunden werden sollen.

Das zum Board gehörende Reference Manual und weitere Anleitungen bzw. Informationen wie z.B. das Master XDC File als Vorlage für die Constraints finden Sie unter:
https://reference.digilentinc.com/reference/programmable-logic/basys-3/start?redirect=1

Weitergehende Information zum eingesetzten FPGA finden Sie bei Xilinx im „Artix-7 Family Overview“, die grundlegenden Leistungsmerkmale finden sich in der folgenden Tabelle aus eben diesem Datenblatt.

Device	Logic Cells	Configurable Logic Blocks (CLBs)		DSP48E1 Slices(2)	Block RAM Blocks(3)			CMTs(4)	PCIe(5)	GTPs	XADC Blocks	Total I/O Banks(6)	Max User I/O(7)
		Slices(1)	Max Distributed RAM (Kb)		18 Kb	36 Kb	Max (Kb)						
XC7A12T	12,800	2,000	171	40	40	20	720	3	1	2	1	3	150
XC7A15T	16,640	2,600	200	45	50	25	900	5	1	4	1	5	250
XC7A25T	23,360	3,650	313	80	90	45	1,620	3	1	4	1	3	150
XC7A35T	33,280	5,200	400	90	100	50	1,800	5	1	4	1	5	250
XC7A50T	52,160	8,150	600	120	150	75	2,700	5	1	4	1	5	250
XC7A75T	75,520	11,800	892	180	210	105	3,780	6	1	8	1	6	300
XC7A100T	101,440	15,850	1,188	240	270	135	4,860	6	1	8	1	6	300
XC7A200T	215,360	33,650	2,888	740	730	365	13,140	10	1	16	1	10	500

Notes:
1. Each 7 series FPGA slice contains four LUTs and eight flip-flops; only some slices can use their LUTs as distributed RAM or SRLs.
2. Each DSP slice contains a pre-adder, a 25 x 18 multiplier, an adder, and an accumulator.
3. Block RAMs are fundamentally 36 Kb in size; each block can also be used as two independent 18 Kb blocks.
4. Each CMT contains one MMCM and one PLL.
5. Artix-7 FPGA Interface Blocks for PCI Express support up to x4 Gen 2.
6. Does not include configuration Bank 0.
7. This number does not include GTP transceivers.

Bild 6 Artix-7 Leistungsmerkmale

Anhang 1: Lösungen und Beispiele für VHDL Code

A.1 Schachuhr

Bitte beachten Sie, dass diese Lösungen nur prinzipieller Natur sind, denn Sie wurden für eine andere Zielhardware (mit z.B. nur drei 7-Segement-Anzeigen) erstellt. Trotzdem können diese wertvoller Input sein und sollten eine gute Grundlagen für ihre eigenen Änderungen darstellen.

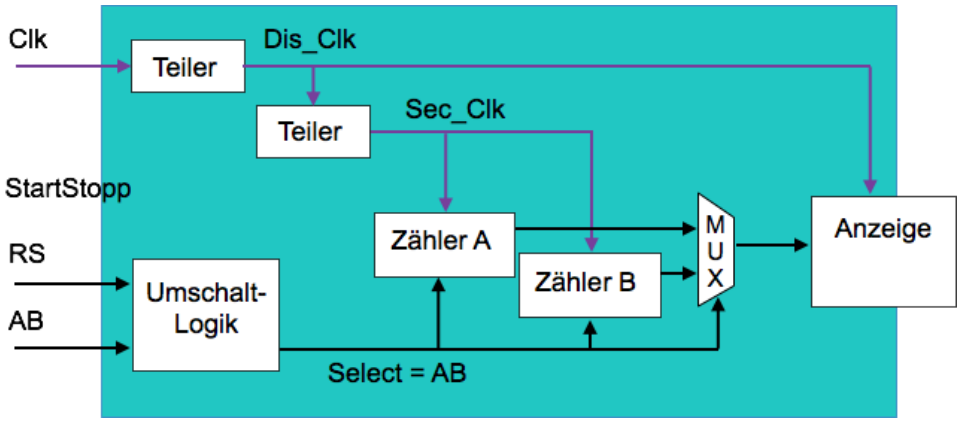
Frage 1.1 Zeichnen Sie ein Blockdiagramm der Schachuhr, das in einer HDL-Beschreibung die Entity darstellt.

Lösung:

Als Eingänge sind vorgesehen: Takt (Clk), Umschaltung zwischen den Spielern A und B (AB), Schalter zum Anhalten bzw. Fortsetzen der Uhr (StartStopp), sowie ein Reset (RS). Gemäß Beschaltung des FPGA wird jeweils eine Ziffer für die 7-Segmentanzeige (DigitOut) im Zeitmultiplex ausgegeben. Der Ausgang MUXOut steuert den Multiplexer bzw. Demultiplexer.

Frage 1.2: Erstellen Sie ein Konzept bzw. grobes Blockschaltbild für das Innenleben: Welche Funktionen bzw. Komponenten benötigen Sie (z.B. Sekundentakt, Speicher bzw. Zähler für die Spielzeiten, Steuerung durch Tasten, Anzeige etc)? Hinweis: Arbeiten Sie vom Groben ins Feine (erst die Struktur, die Details später). Ignorieren Sie zunächst bitte Reset und Start/Stopp. Abstrahieren Sie die Ansteuerung der 7-Segmentanzeige zu einem Funktionsblock Anzeige, der später detailliert werden kann.

Lösung: z.B. wie folgt.

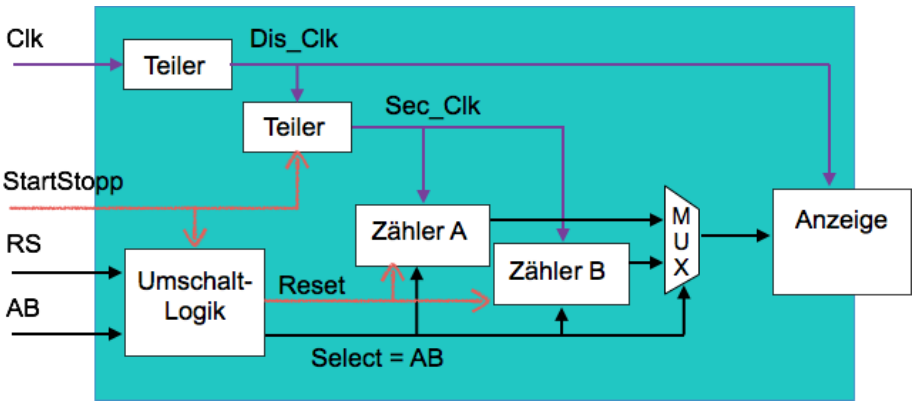


Die Schaltung benötigt zum Zählen der Sekunden für die Spieler A und B einen Sekundentakt (Sec_Clk). Zum Ausgeben der 3 Ziffern auf die 7-Segmentanzeige wird außerdem ein weiteres Taktsignal benötigt (Dis_Clk). Beide Signale werden durch Teiler aus dem Taktsignal (Clk) erzeugt.

Kern der Schaltung sind die beiden Zähler für die von den Spielern A und B jeweils benötigte Zeit (Zähler A und Zähler B). Mit Hilfe des Umschalters AB wird zwischen diesen Zählern umgeschaltet. Der jeweils aktive Zähler wird über einen Multiplexer zur Anzeige gebracht.

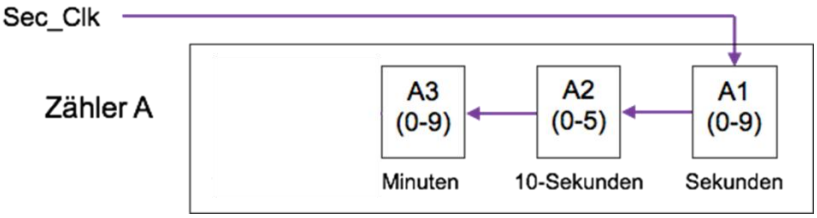
Frage 1.3: Ergänzen Sie das Blockschaltbild um die Funktionen Reset und Start/Stopp. Begründen Sie Ihren Entwurf.

Lösung: z.B. wie folgt. Das Signal StartStopp hält den Sekundentakt an. In der hier gewählten Lösung lässt sich die Schaltung nur bei angehaltener Uhr zurücksetzen (Signal RS). Das Zurücksetzen löscht beide Zählerstände A und B. Bemerkung: Reset sollte durch den Prozess ausgeführt werden, der die Zählerstände erzeugt (andernfalls Zugriffskonflikte).



Frage 1.4: Detaillieren Sie die Speicher bzw. Zähler für die Spielstände. Begründen Sie Ihren Entwurf.

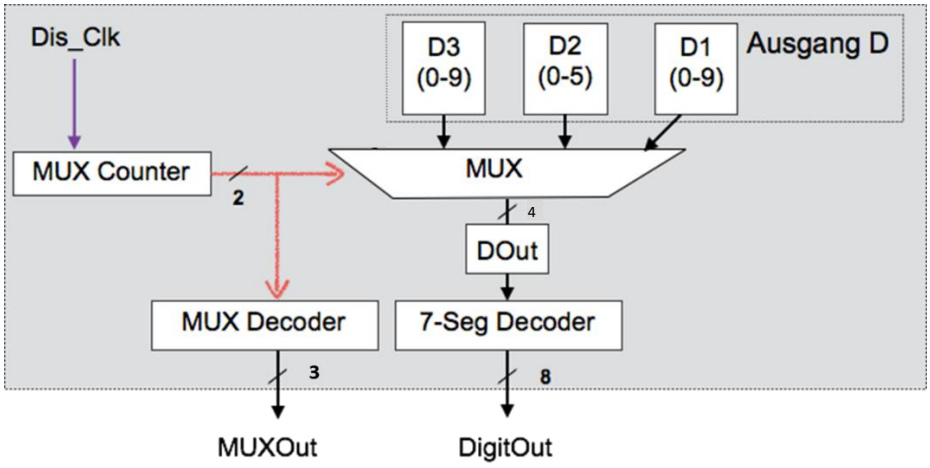
Lösung: Die Zähler lassen sich z.B. gleich mit Hilfe jeweils einer Ziffer pro Dezimalstelle ausführen, d.h. Ziffer1 für die Sekunden (von 0 bis 9), Ziffer 2 für 10-Sekunden (von 0 bis 5), Ziffer 3 für die Minuten (von 0 bis 9). Ziffer 4 für 10-Minuten (von 0 bis 5) war mit der früherzur Verfügung stehenden HW nicht möglich, die hier angegebene Lösung beschränkt sich also auf eine Anzeige bis 9 Minuten 59 Sekunden. Eine Erweiterung können Sie aber leicht selber vornehmen. Überlauf einer Stelle löst einen Zählimpuls in der folgenden Stelle aus.



Mit dem Signal (AB) für das Umschalten zwischen den Spielern A und B wird jeweils ein Zähler für A und B aktiviert, sowie der Ausgang des Jeweils aktiven Zählers auf Register D1 bis D4 zur weiteren Verarbeitung übertragen (Funktion des Multiplexers im Blockschaltbild).

Frage 1.5: Detaillieren Sie den Funktionsblock für die 7-Segmentanzeige so, dass hervorgeht, wie Sie die in der Abbildung oben gezeigten 3 Ziffern ansteuern.

Lösung: z.B. wie in folgendem Diagramm. Der Ausgang D, der den jeweils aktiven Spieler repräsentiert, wird ziffernweise ausgegeben. Der Zeitmultiplex erfolgt im Takt der Display-Clock (Dis_Clk). Für die Ausgabe auf der Beschaltung des FPGAs wird das Multiplex-Signal in 3 Bits kodiert (Ausgangssignal MUXOut), sowie die jeweils übertragene Ziffer vom BDC-Format in das Format der 7-Segmentanzeige (Ausgangssignal DigitOut).



Frage 1.6: Übersetzen Sie das Blockdiagramm in HDL-Text für den Architektur-Block (Struktur mit Prozessen, ohne Details).

Lösung: z.B. wie im folgenden HDL-Text. Zum Testen auf dem Simulator sind große Teiler für den Takt nicht praktikabel, daher wurde auf den Teiler für die Display-Clock verzichtet. Dieser kann bei einer Synthese ergänzt werden. Teiler für eine 8 MHz Taktrate wäre: (1) 80000 für eine Display Clock (Dis_Clk) von 100 Hz, (2) 100 zur Ableitung des Sekundentaktes (Sec_Clk) aus der Display-Clock (DIS_Clk). (Bitte Beachten Sie, das der VHDL Code teilweise noch angepasst werden muss!)

Struktur der Prozesse: entsprechend dem Blockschaltbild

(1) Teiler (Zähler) zur Erzeugung des Takts für Display (z.B. 100 Hz) und davon abgeleitet für die Sekunden (1 Hz)

(2) Zähler für die Spieler A und B mit Umschaltlogik und Multiplexer für den aktiven Spieler (auf die Ausgangssignale D1 bis D4). Die Zähler sind gleich als Dezimalstellen ausgeführt. Dieser Prozess wird mit dem Sekundentakt (Sec_Clk) betrieben.

(3) Zähler für den Multiplexer bzw. Demultiplexer für die Anzeige (Select Signal MUX). Dieser Prozess wird mit dem Takt für das Display betrieben (Dis_Clk).

(4) Anzeigensteuerung: Der Zähler MUX (Select-Signal) wird übersetzt in 4 einzelne Bits (Signal Display_MUX). Dieser Prozess folg dem MUX-Zähler (MUX).

(5) Dekoder für 7-Segmentanzeige: In der hier gewählten Lösung sitzt der Dekoder in der Multiplex-Leitung, d.h. die jeweils übertragene Ziffer DOut (im BCD-Format) wird übersetzt in das 7-Segment-Format DigitOut. Dieser Prozess folgt dem Signal DOut.

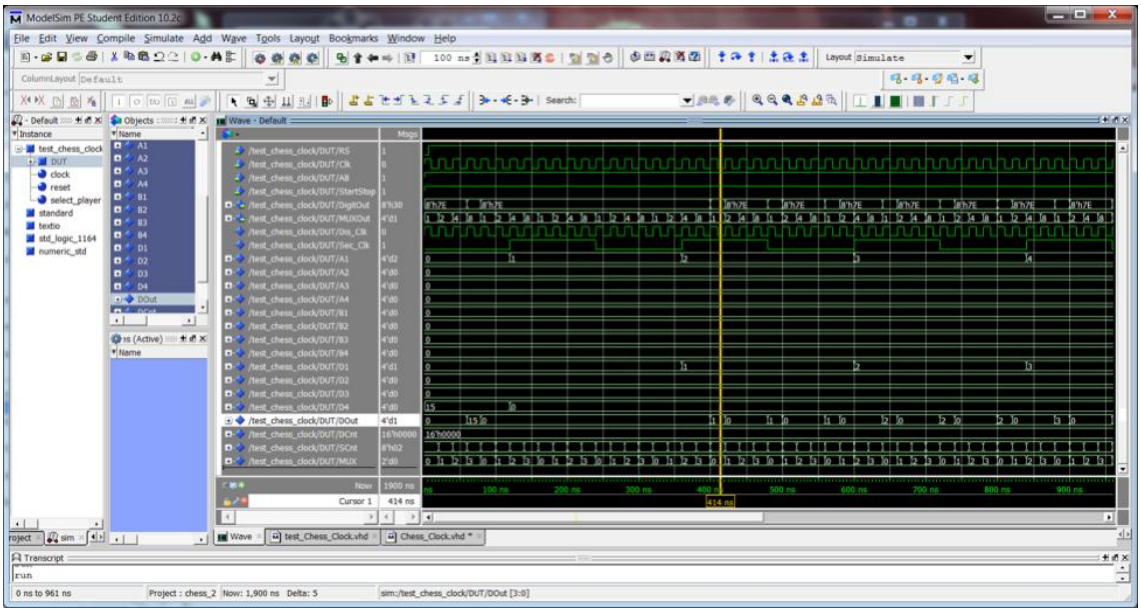
```
-----------------------------------------------------------------------------
--- Chess_Clock
--- Solution for MIMAS V2 Board from Numato
--- Author: Klaus Gosger
---
--- Please note: this code needs adaptation when used on other boards!
-----------------------------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.numeric_std.all;

entity Chess_Clock is
```

```
 Port (RS         : in STD_Logic;     -- Reset (active low)
       Clk        : in  STD_LOGIC;    -- Serial clock ? MHz
       AB         : in  STD_LOGIC;    -- DP Switch
       StartStop  : in  STD_LOGIC;    -- Stop Clock = 0, Continue = 1
       DigitOut   : out  STD_LOGIC_vector (7 downto 0); -- One Segment
       MUXOut     : out  STD_LOGIC_vector (2 downto 0));-- Select Segemnt
end Chess_Clock;

architecture RTL of Chess_Clock is

-- internal signals and variables
signal Dis_Clk : std_Logic := '0'; -- display clock
signal Sec_Clk : std_Logic := '0'; -- clock in seconds
signal D1, D2, D3, DOut : std_logic_vector (3 downto 0):= (others => '0');
        -- minutes (D3) and seconds (D2_1) as output to the display
signal DCnt : unsigned (15 downto 0):= (others=>'0');  -- counter for Dis_Clk
signal SCnt : unsigned (7 downto 0):= (others=>'0');   -- counter for Sec_Clk
signal MUX  : unsigned (1 downto 0):= (others=>'0');   -- Select Signal for
MUX and Decoding of Digits

signal A1, A2, A3 : unsigned (3 downto 0):= (others=>'0');
    -- minutes (4_3) and seconds (2_1) Player A
signal B1, B2, B3 : unsigned (3 downto 0):= (others=>'0');
    -- minutes (4_3) and seconds (2_1)  Player B

begin

Dis_Clk <= Clk; -- consider Clk as Display clock for tests in simulation

  -- clock dividers for Dis_Clk and Sec_Clock
  Seconds : process (Dis_Clk, StartStop) begin
    if (rising_edge(Dis_Clk) and (StartStop = '1')) then
         if (SCnt < 50) then SCnt <= SCnt + 1;
          else
              SCnt <= "00000000";
              Sec_Clk <= not Sec_Clk;
          end if;
      end if;
  end process Seconds;

  -- counter for players A and B
  AB_count : process (Sec_Clk, AB, StartStop, RS)
  begin
       case AB is
       when '1' =>  -- counting for A
            if (rising_edge(Sec_Clk) and StartStop = '1') then
                  if (A1 < "1001") then
                         A1 <=(A1 + 1);
                    else
                         A1 <= "0000";
```

```
                        if A2 < "0101" then
                             A2 <= (A2 + 1);
                             else A2<="0000";
                               if A3 < "1001" then
                               A3 <= (A3 + 1);
                               else A3 <="0000";
                               end if;
                        end if;
                  end if;
          end if;
  when others =>   -- counting for B
          if (rising_edge(Sec_Clk) and StartStop = '1') then
                if (B1 < "1001") then
                        B1 <=(B1 + 1);
                  else
                        B1 <= "0000";
                        if B2 < "0101" then
                             B2 <= (B2 + 1);
                             else B2<="0000";
                               if B3 < "1001" then
                               B3 <= (B3 + 1);
                               else B3 <="0000";
                               end if;
                        end if;
                  end if;
          end if;
  end case;
 end process AB_count;

-- output mux
 AB_out : process (A1,A2,A3,B1,B2,B3,AB)
 begin
   case AB is
        when '1' =>                    -- output for A
          D1 <= std_logic_vector(A1);
          D2 <= std_logic_vector(A2);
          D3 <= std_logic_vector(A3); -- output counter A
        when others =>                    -- output for B
          D1 <= std_logic_vector(B1);
          D2 <= std_logic_vector(B2);
          D3 <= std_logic_vector(B3); -- output counter B
    end case;
 end process AB_out;

-- multiplexer select signal for 7-segment display
MUXCount : process (Dis_Clk) begin
  if (rising_edge(Dis_Clk)) then
      if (MUX < 2) then MUX <= MUX + 1;
        else MUX <= "00";
      end if;
```

```
    end if;
  end process MUXCount;

  -- multiplex and decode to 7-segment display
  Display_MUX : process (MUX, Dis_Clk) -- MUX Decoder
  begin
    if rising_edge(Dis_Clk) then
      case MUX is
            when "00" => MUXOut <= "110"; -- drive segment S3 via Enable 3
            when "01" => MUXOut <= "101"; -- drive segment S2 via Enable 2
            when "10" => MUXOut <= "011"; -- drive segment S1 via Enable 1
            when others => MUXOut <= "111";
          end case;
     end if;
  end process Display_MUX;

  DOut_MUX : process (MUX, Dis_Clk,D1, D2, D3) -- MUX generating DOut
  begin
    if rising_edge(Dis_Clk) then
      case MUX is
            when "00" => DOut <= D1;
            when "01" => DOut <= D2;
            when "10" => DOut <= D3;
            when others => NULL;
        end case;
    end if;
  end process DOut_MUX;

  Display_Decoder : process (DOut) -- decodes DOut to 7-Segm.
  begin
    case DOut is            --  pabcdefg
      when x"0" => DigitOut <= "10000001";
      when x"1" => DigitOut <= "11001111";
      when x"2" => DigitOut <= "10010010";
      when x"3" => DigitOut <= "10000110";
      when x"4" => DigitOut <= "11001100";
      when x"5" => DigitOut <= "10100100";
      when x"6" => DigitOut <= "00100000"; -- punkt an
      when x"7" => DigitOut <= "10001111";
      when x"8" => DigitOut <= "10000000";
      when x"9" => DigitOut <= "00000100"; -- punkt an
      when others => DigitOut <= "00110000"; -- E: Error
    end case;
  end process Display_Decoder;

end RTL;
```

Zur Ergänzung ein Simulatorlauf (mit SCnt < 5 statt 50):



A.2 Muster für Verifikationstabelle

Die folgende Tabelle stellt die verschiedenen durchgeführten Testfälle inklusive der dazugehörigen Beschreibung des Testfalls dar. Zusätzlich wird das erwartete Ergebnis mit dem Testergebnis verglichen und entsprechend markiert, ob der Test bestanden wurde.

Testfall	Testbeschreibung	Erwartetes Ergebnis	Testergebnis	Bestanden
1	**Ausgangszustand:** direkt nach Anlegen der Stromversorgung Beide Zähler auf Startwert **Benutzerinterface:** Spielerwahlschalter auf A StartStop-Schalter auf aus Reset nicht betätigt	Anzeige von Startwert von Spieler A auf 7-Segment-Anzeige, alle LEDs aus	Anzeige von Startwert von Spieler A auf 7- Segment-Anzeige, alle LEDs aus	☑
2	**Ausgangszustand:** direkt nach Anlegen der Stromversorgung Beide Zähler auf Startwert **Benutzerinterface:** Spielerwahlschalter auf A StartStop-Schalter auf aus Reset nicht betätigt	Anzeige von Startwert von Spieler B auf 7-Segment-Anzeige, alle LEDs aus	Anzeige von Startwert von Spieler B auf 7-Segment-Anzeige, alle LEDs aus	☑
3	**Ausgangszustand:** direkt nach Anlegen der Stromversorgung Beide Zähler auf Startwert **Benutzerinterface:** Spielerwahlschalter auf A StartStop-Schalter auf ein Reset nicht betätigt	Zähler von Spieler A zählt runter, Spieler B bleibt unverändert, alle LEDs aus	Zähler von Spieler A zählt runter, Spieler B bleibt unverändert, alle LEDs aus	☑
usw…				

A.3 Entprellen

Häufig sind die Taster und Schalter auf einem Demo-Board nicht entprellt. Grundsätzlich muss davonausgegangen werden, dass Eingangssignale mit Störungen beaufschlagt sein können, so dass es sich empfiehlt sicherzustellen, dass das Eingangssignal nach einer Änderung des Pegels den aktuellen Wert für eine gewisse Zeit ohne Änderung hält.

In der Literatur und im Internet finden Sie Beispiele für eine mögliche Entprellschaltungen, hier ein Vorschlag aus dem Buch von C. Elias (2):

```
-- debounce.vhd

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity debounce is
generic( num_cons_cycles : integer := 2000;
         cntr_size : integer := 11);
port(clk : in std_logic;
     data_in : in std_logic;
     data_deb : out std_logic );
end debounce;

architecture impl of debounce is

signal cntr_reg : unsigned(cntr_size-1 downto 0) := (others => '0');
signal cntr_next : unsigned(cntr_size-1 downto 0);

signal syncbuf_reg : std_logic_vector(2 downto 0) := (others => '0');
signal syncbuf_next : std_logic_vector(2 downto 0);

signal deb_reg : std_logic;
signal deb_next : std_logic;

begin

reg_proc : process (clk)
begin
  if rising_edge(clk) then
    syncbuf_reg <= syncbuf_next;
    cntr_reg <= cntr_next;
    deb_reg <= deb_next;
  end if;
end process reg_proc;

syncbuf_next <= syncbuf_reg(1 downto 0) & data_in;
```

```
deb_proc : process(syncbuf_reg(2 downto 1), cntr_reg, deb_reg, data_in)
begin
  deb_next <= deb_reg; -- default
  cntr_next <= cntr_reg; -- default
  if syncbuf_reg(1) = syncbuf_reg(2) then -- sync. data stable?
    if cntr_reg = num_cons_cycles then
      deb_next <= syncbuf_reg(2);
    else
      cntr_next <= cntr_reg + 1;
    end if;
    else
      cntr_next <= (others => '0');
   end if;
end process deb_proc;

data_deb <= deb_reg;

end impl;
```

Anhang 2: Literatur & SW

(1) Klaus Gosger, Entwurf digitaler Systeme, Vorlesungsmanuskript, DHBW-Stuttgart, 2025

(2) Cord Elias, FPGAs für Maker, dpunkt.verlag, 2016

(3) Entwicklungswerkzeug: Xilinx Vivado Design Suite - HLx Editions (Installation via Self Extracting Web Installer) mit Digilent Board Files (bitte vorab auf Ihrem Laptop installieren):

Vorgehen siehe hier:
https://digilent.com/reference/programmable-logic/guides/installing-vivado-and-vitis

Vivado gibt es hier:
https://www.xilinx.com/support/download/index.html/content/xilinx/en/downloadNav/vivado-design-tools.html

(4) Master XDC File zur Festlegung der Constraints des Zielsystems
https://github.com/Digilent/digilent-xdc/

(5) Zielsystem: Basys 3™ FPGA Board von Digilent, Reference Manual und weitere Anleitungen bzw. Informationen siehe:
https://reference.digilentinc.com/reference/programmable-logic/basys-3/start?redirect=1

(6) Artix-7 FPGA Family Overview:
https://www.xilinx.com/support/documentation/data_sheets/ds180_7Series_Overview.pdf